

Lab 8

In this lab, you'll be parallelizing a program to count prime numbers. (These are the positive integers that are only divided without remainder by themselves and one.)

The Fork-Join Framework

Begin by downloading the given code (`lab8given.zip`). Unzip this and open the folder `lab8-primes` it contains as a Java project.

This project contains several programs (in the `src` subfolder). Begin by looking at `SumArray.java`, which is the program to sum an array using the fork-join framework. This uses the actual Java version of the framework instead of the convenient `fork` and `join` keywords we used in class. A couple of things to notice:

- It uses a class `SumArray` to represent a piece of work, basically a subarray to sum. This class extends `RecursiveTask<Integer>`, with the `Integer` being the type of value returned by the computation. (Java has features that let us pretend that it returns `int`.)
- The work is done by the `compute` method, which can't take arguments. Thus, the part of the array to sum is controlled by the `hi` and `low` attributes, which are set in the constructor.
- The last paragraph of code in `main` initiates the computation. It puts a single task responsible for summing the entire array into a "pool" of tasks to be completed. This is a collection of tasks that the processors will draw from whenever they are idle. Tasks created during the recursion are automatically added to the same pool.

Look at all of these to understand how to structure computations in this framework. (It's definitely more complicated than the `fork` and `join` keywords, but you can refer back to this version as needed.)

Once you're ready, you're going to apply the fork-join framework to a new problem. This problem will be counting prime numbers, those that are only exactly divisible by themselves and 1. (The first several prime numbers are 2, 3, 5, 7, 11, 13.) `SerialPrimes.java` contains a serial program to count the number of primes between 3 and 2,000,000. We're not counting 2 because it's the only even prime number. For the odd prime numbers, we're going to represent them in the form $2 * \text{num} + 1$ where $1 \leq \text{num} < 999,999$. By working in terms of `num` rather than the number itself, our loop can use contiguous indices and we know that all the potential primes being considered are odd. The use of contiguous indices simplifies our parallelization of this program.

Once you're comfortable with this code, use the fork-join framework to parallelize `ForkJoinPrimes.java`, which (other than the name) is identical to `SerialPrimes.java`. You'll need to make `ForkJoinPrimes` extend `RecursiveTask<Integer>` or add a class that does so. Replace the loop in `main` with a call to `ForkJoinPool.commonPool().invoke` that runs one of your task objects. Give your task class a `compute` method that returns the number of primes in its range. (Use recursion to split the problem, but calling `isPrime` for the actual testing in its base case). Remember that `RecursiveTask` objects return the value they calculate.

One question that remains is what to use as a base case for the recursion; `SumArray` stops recursing when its part of the array is size 100, but I want you to try different values for this class. In particular, try 500,000, which will stop the recursion after creating 2 tasks. Also try 250,000 (4 tasks) and some smaller values. How does the final task size affect the running time? (You probably want to make multiple runs at each size since there may be some random variability.) If you have more time, modify your code to split the array into 3 parts instead of 2 so that you can try using 3 tasks. (Note that you can't just change the threshold to achieve this because the recursive calls split tasks into pairs.)